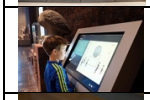
INTRODUCTION

In your role as a software developer, you have been contracted by the Ulster Museum to design, implement and test a Java application capable of managing a collection of exhibitions that will be shown in the museum throughout the year. The exhibitions are made up of artefacts that they have stored in their collection. Each artifact has a type, such as artwork, statues, interactive exhibits etc. the goal is to create a curated route through the exhibit showcasing the artifacts and conveying a compelling experience. The route should include text for the signs connecting the artifacts together into a coherent message. Each artifact has an expected engagement time that estimates how long visitors will spend interacting with it. There are 3 exhibition halls and the exhibits change each month.

Example Artifacts:

Image	*Name*	*Type*	*Engagement Time*
	Delaware Landscape	Painting	1min
	Acropolis Statues	Sculpture	5mins
	Incas Interactive	Digital	10mins
	TouchIt	Tactile	8mins

Example Exhibit: Global Warming

Artifacts for the Exhibit:

1KG Carbon Statue
Coal Miners Portrait
Dinosaur Statue
Home Carbon Footprint Interactive
Tar Pit Miniature

Route:

Steps	**Signs**
1. 1KG Carbon Statue	"1KG of Carbon: Small in size, big in impact. Imagine this much carbon released every time you drive 6 miles."
2. Dinosaur Statue	"Ancient Giants: Once thriving, now fossils. Climate change contributed to past extinctions—will history repeat itself?"
3. Tar Pit Miniature	"Fossil Fuel Origins: Once a lush habitat, now trapped in tar. What are we trapping in our atmosphere today?"
4. Coal Miners Portrait	"Fueling the Industrial Age: Coal miners powered progress. Now, can we power progress sustainably?"
5. Home Carbon Footprint Interactive	"Calculate Your Impact: Explore ways to reduce your footprint at home. Small changes add up!"

Example Exhibition Plan:

	Exhibit Hall 1	***Exhibit Hall 2***	***Exhibit Hall 3***
January	Global Warming	Vikings	Henry VIII
February	Egypt	History of AI	Geology
March	Architecture	Ship Building	Beasts of the Sea
…	…	…	…

GENERAL INSTRUCTIONS

1. Create a new Java project named 'Assessment2'.
2. Add three packages to 'Assessment2' named part01, part02 and part03.
3. Implement your code as described below:
    - 'part01' should contain all code associated with the core requirements.
    - 'part02' should contain any code relating to testing.
    - 'part03' should contain the code related to implementation of additional features only, including the use of class **Console**, introduced in the material for reading week.

SUBMISSION DATE/TIME – MONDAY 6TH DECEMBER 2024 BY 5 PM.

PART 1: CORE FUNCTIONALITY (50 MARKS AVAILABLE)

To demonstrate that the proposed Java application is feasible, the following must be implemented.

1.1 Development of a **UML diagram** to describe the components of the system (object classes, enumerations, interfaces and their associations) necessary to model the scenario described above. **Note:** You do not need to consider any application class in this UML diagram.

1.2 Implementation of the system components in **Java**, in line with the UML diagram produced in 1.1.

1.3 A standard text console (menu-based) application (called ***QUBMuseum***) to provide system behaviour. The following general menu options should be provided:

- ***Manage Artifacts***: the user should be able to add, view, delete and update artifacts used within the system.
- ***Manage Exhibits***: the user should be able add, view, delete and update exhibits used within the system.
- ***Manage Annual Plan***: the user should be able to create, view and modify the annual plan of exhibits within the system.
- ***Exit***: the user should be able to exit the system.

Notes:

A. Artifacts and exhibits should be uniquely identifiable within the system.

B. The system application should make use of the standard console **Menu** class described in the lectures (and made available on Canvas).

C. When viewing artifacts and exhibits, the user should be presented with a list sorted in alphabetical order.

D. When viewing artifacts and exhibits, the user should be able to search by varied criteria (by unique id, by name, by part-name, by type)

E. Overall engagement time of an exhibit should be presented when viewing.

F. By default, the system application should provide sufficient artifacts and exhibits to fulfil an annual schedule.

G. You are restricted to **Java core language features** described at the start of the module and used in the code examples.

H. You may use **Scanner** and **ArrayList** (from java.util - restricted to **add**, **get, size** and **remove** methods only).

I. You are not permitted use of any other library.

PART 2: TESTING THE APPLICATION (20 MARKS)

As part of the Ulster Museum's quality assurance procedure, you are required to submit evidence that **unit** and **integration** tests have been designed and applied. The following deliverables are required:

1. a completed **testing document**,
2. supporting **test code**
3. evidence of **test execution outcome**.

The scope of this testing should cover all areas of core functionality outlined in **Part 1** of this assignment specification only. Testing documentation must be submitted using the template provided by QUB, which will be made available on Canvas and will be discussed in the forthcoming lectures.

PART 3: ADDITIONAL FUNCTIONALITY (30 MARKS)

3.1 Either:

Use the ***Console*** class to implement a new application called ***QUBMediaMuseum*** which replicates the behaviour of ***QUBMuseum*** from ***Part 1***. Specifically, the application should:

- Make use of a ***Console*** instance for all input and display functionality.
- Use images as appropriate to enhance the user experience.
- Apply the same coding restrictions outline for Part 1 above, aside from using **CSC1027Console.jar** containing the definition of class **Console.** You can also make use of any unassessed code shared with the class, such as the JSON reading and writing sample code.

Or

Use the ***WebInterface*** class to implement a new application called ***QUBWebMuseum*** which replicates the behaviour of ***QUBMuseum*** from ***Part 1***. Specifically, the application should:

- Make use of a ***WebInterface*** instance for all input and display functionality.
- Use HTML (and optionally javascript) as appropriate to enhance the user experience.
- Apply the same coding restrictions outline for Part 1 above, aside from using **WebInterface** code. You can also make use of any unassessed code shared with the class, such as the JSON reading and writing sample code.

Marks will be allocated for functionality/behaviour and creativity in the use of the ***Console*** or ***WebInterface*** classes.

CHATGPT OR OTHER AI

You are required to provide a brief report indicating where and how you have used ChatGPT or any other AI in your work for this assessment.
Include the following information for part 1, 2 and 3:

1. Version of ChatGPT/AI used,
2. Details of use (e.g. for code generation, debugging)

If have not used ChatGPT/AI in any way – please state this in your (very brief) report.

SUBMISSION INSTRUCTIONS

1. Create a new folder to hold all your assignment files for submission. The folder should be suitably named and include your student number.

2. Copy and paste the 'part01', 'part02' and 'part03' folders from your java project into the folder created in step 1. ENSURE that the 'part01', 'part02' and 'part03' folders contain all of the corresponding .java files.
3. Include the UML diagram (for Part 1) and the Images folder (used in Part 3)
4. Add your testing spreadsheet and test output evidence to the folder created in step 1. Make sure to include your test code (for Part 2)
5. Verify that you have all the required files included in the folder created in step 1.
6. Compress the folder to a zip file and upload it to the assignment location in the CSC1027 module on Canvas by **5.00pm** on **Monday 6th December 2024.**

NOTE: Please check that you have included the original development Java (.java) files. If you submit .class files these will NOT be marked. These submissions will be date-stamped and in accordance with University regulations, late submissions will be penalised. **Failure to follow the submission instructions may result in a reduced or zero mark for part or all of the assignment!**

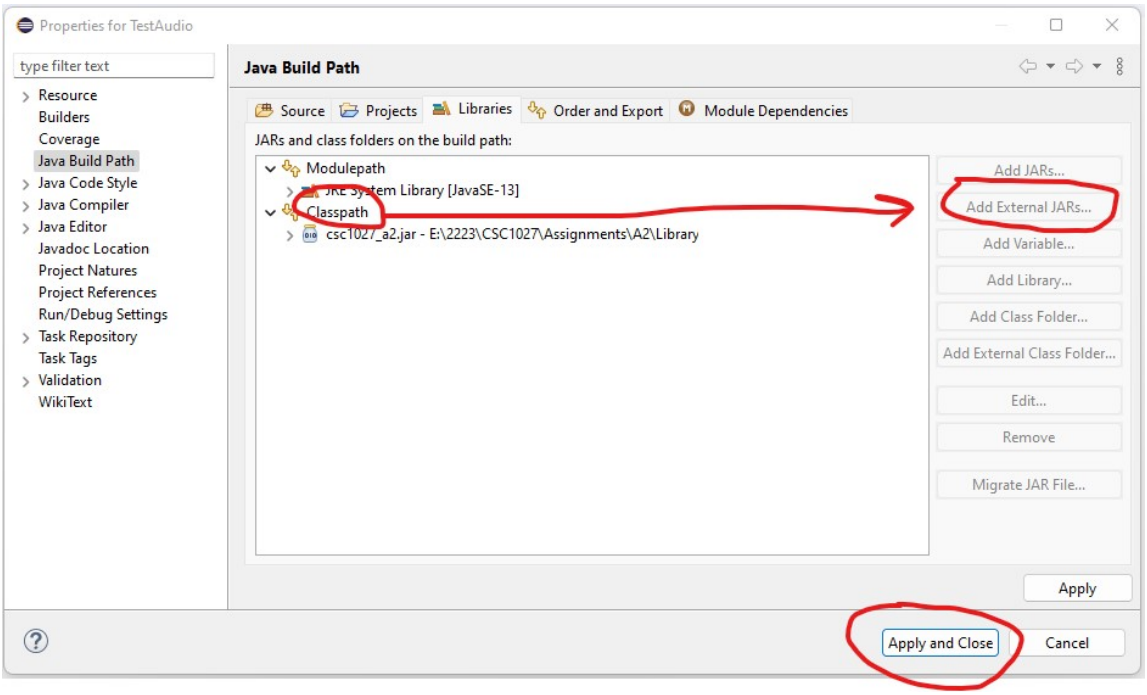
APPENDIX 1: HOW TO INSTALL THE EXTERNAL LIBRARY FILE (JAR)

Step 1 – Download the following files from Canvas (under Assessment 2)

- ***CSC1027Console.jar*** – this file contains the definitions for classes ***Console***

Step 2 – Add the downloaded jar file to your project:

- From the package explorer in Eclipse, "***right-click***" your project directory and select "***Build Path->Configure Build Path ..***"
- Select the "***Libraries***" tab and highlight "***Classpath***"
- Click the "***Add External Jar***" button (on the right-hand side) and navigate to the downloaded Jar file to select/open.
- Click on the "***Apply and Close***" button



<-- Your Package Explorer should look something like this

<-- The Jar file should be visible within "Referenced Libraries"